



Rango V2.1

Security Audit Report

June 15, 2026

Contents

1	Introduction	3
1.1	About Rango	3
1.2	Source Code	3
1.3	Revision History	3
2	Overall Assessment	4
3	Vulnerability Summary	5
3.1	Overview	5
3.2	Security Level Reference	6
3.3	Vulnerability Details	7
3.3.1	[H-1] Signature Replay in Permit2Proxy::permitAndCallDiamond()	7
3.3.2	[H-2] OApp and bridgeToken Mismatch in lzCompose()	8
3.3.3	[M-1] Possible Revert in LibSwapper::callSwapsAndFees()	9
3.3.4	[M-2] Improper Swap-out Amount in onChainSwapsInternal()	11
3.3.5	[M-3] Revisited Logic of LibSwapper::onChainSwapsPreBridge()	12
3.3.6	[M-4] Lack of Input Validation in callRecieveMessage()	13
3.3.7	[M-5] Reentrancy Guard Bypass Between Old and New Facets	14
3.3.8	[L-1] Potential Risks Associated with Centralization	15
3.3.9	[L-2] Improper Event in _handleUniswapV2()/ _handleCurve()	16
3.3.10	[L-3] Revisited Logic of LibSwapper2::returnExcessAmounts()	17
3.3.11	[L-4] Incompatible msg.sender Fallbacks with Permit2Proxy Call Path	18
4	Appendix	21
4.1	About AstraSec	21
4.2	Disclaimer	21
4.3	Contact	21

1 | Introduction

1.1 About Rango

Rango is a new layer on top of all Bridges and DEXs, working as a Bridge Aggregator and DEX Aggregator at the same time to enable seamless on-chain and cross-chain swaps, finding the most efficient, safe, cheap and fast route for swapping from any token on any blockchain to any other token to any blockchain.

1.2 Source Code

The following source code was reviewed during the audit:

- <https://github.com/rango-exchange/rango-contracts-v2>
- CommitID: ec28303

And this is the final version representing all fixes implemented for the issues identified in the audit:

- <https://github.com/rango-exchange/rango-contracts-v2>
- CommitID: 246816c

1.3 Revision History

Version	Date	Description
v2.1.0	June 26, 2024	Initial Audit
v2.1.1	July 15, 2025	Generic Bridge and CCTPV2 Middleware
v2.1.2	June 1, 2026	Add OFT Middleware, Permit2 Proxy, etc.

2 | Overall Assessment

This report has been compiled to identify issues and vulnerabilities within the Rango V2.1 protocol. Throughout this audit, we identified a total of 11 issues spanning various severity levels. By employing auxiliary tool techniques to supplement our thorough manual code review, we have discovered the following findings.

Severity	Count	Acknowledged	Won't Do	Addressed
Critical	-	-	-	-
High	2	-	-	2
Medium	5	1	-	4
Low	4	2	-	2
Informational	-	-	-	-
Undetermined	-	-	-	-

3 | Vulnerability Summary

3.1 Overview

Click on an issue to jump to it, or scroll down to see them all.

- [H-1](#) Signature Replay in Permit2Proxy::permitAndCallDiamond()
- [H-2](#) OApp and bridgeToken Mismatch in IzCompose()
- [M-1](#) Possible Revert in LibSwapper::collectFeesForSwap()
- [M-2](#) Improper Swap-out Amount in LibSwapper::onChainSwapsInternal()
- [M-3](#) Revisited Logic of LibSwapper::onChainSwapsPreBridge()
- [M-4](#) Lack of Input Validation in RangoCCTPV2Middleware::callRecieveMessage()
- [M-5](#) Reentrancy Guard Bypass Between Old and New Facets
- [L-1](#) Potential Risks Associated with Centralization
- [L-2](#) Improper Event in LibInterchain::_handleUniswapV2()/ _handleCurve()
- [L-3](#) Revisited Logic of LibSwapper2::returnExcessAmounts()
- [L-4](#) Incompatible msg.sender Fallbacks with Permit2Proxy Call Path

3.2 Security Level Reference

In web3 smart contract audits, vulnerabilities are typically classified into different severity levels based on the potential impact they can have on the security and functionality of the contract. Here are the definitions for critical-severity, high-severity, medium-severity, and low-severity vulnerabilities:

Severity	Description
C-X (Critical)	A severe security flaw with immediate and significant negative consequences. It poses high risks, such as unauthorized access, financial losses, or complete disruption of functionality. Requires immediate attention and remediation.
H-X (High)	Significant security issues that can lead to substantial risks. Although not as severe as critical vulnerabilities, they can still result in unauthorized access, manipulation of contract state, or financial losses. Prompt remediation is necessary.
M-X (Medium)	Moderately impactful security weaknesses that require attention and remediation. They may lead to limited unauthorized access, minor financial losses, or potential disruptions to functionality.
L-X (Low)	Minor security issues with limited impact. While they may not pose significant risks, it is still recommended to address them to maintain a robust and secure smart contract.
I-X (Informational)	Warnings and things to keep in mind when operating the protocol. No immediate action required.
U-X (Undetermined)	Identified security flaw requiring further investigation. Severity and impact need to be determined. Additional assessment and analysis are necessary.

3.3 Vulnerability Details

3.3.1 [H-1] Signature Replay in Permit2Proxy::permitAndCallDiamond()

Target	Category	IMPACT	LIKELIHOOD	STATUS
Permit2Proxy.sol	Business Logic	High	High	Addressed

`permitAndCallDiamond()` allows a user to sign a `diamondCalldata` payload off-chain, then have a relayer execute it on their behalf. The signature verification uses an EIP-712 typed data digest constructed from `CALLDATA_WITNESS_TYPEHASH`, `owner`, `token`, `amount`, and `keccak256(diamondCalldata)`. The struct hash does not include a `nonce` or `deadline` field specific to the `calldata` signature. While the function accepts a `deadline` parameter, that `deadline` is only used for the underlying ERC20 `permit()` call, not for the `calldataSignature` validation. The EIP-712 digest for `calldataSignature` remains identical for every invocation with the same `(owner, token, amount, diamondCalldata)` tuple. Once a user signs a `calldata` payload, that signature is valid forever (or until the token's `permit` allowance is consumed). An attacker or malicious relayer can replay the same `calldataSignature` multiple times to repeatedly execute the same diamond call.

Permit2Proxy::permitAndCallDiamond()

```
81 function permitAndCallDiamond(  
82     address owner,  
83     address token,  
84     uint256 amount,  
85     uint256 deadline,  
86     uint8 v,  
87     bytes32 r,  
88     bytes32 s,  
89     bytes calldata diamondCalldata,  
90     bytes calldata calldataSignature  
91 ) external payable returns (bytes memory) {  
92     if (token == address(0)) revert InvalidTokenAddress();  
  
94     // Verify the owner signed the calldata  
95     bytes32 structHash = keccak256(abi.encode(  
96         CALLDATA_WITNESS_TYPEHASH,  
97         owner,  
98         token,  
99         amount,  
100         keccak256(diamondCalldata)  
101     ));  
102     bytes32 digest = _hashTypedDataV4(structHash);  
103     address recovered = ECDSA.recover(digest, calldataSignature);  
104     if (recovered != owner) revert InvalidCalldataSignature();
```

```

106     // Execute permit (skip if allowance already sufficient)
107     IERC20 erc20 = IERC20(token);
108     if (erc20.allowance(owner, address(this)) < amount) {
109         IERC20Permit(token).permit(owner, address(this), amount, deadline, v, r,
110             s);
111     }

112     erc20.safeTransferFrom(owner, address(this), amount);

114     _maxApproveDiamond(token);

116     return _callRangoDiamond(diamondCalldata);
117 }

```

Remediation Add nonce and deadline fields to CALLDATA_WITNESS_TYPEHASH and include them in the struct hash. Maintain a mapping of owner => nonce in Permit2Proxy and increment it after each successful call. Reject signatures with a stale or reused nonce or an expired deadline.

3.3.2 [H-2] OApp and bridgeToken Mismatch in lzCompose()

Target	Category	IMPACT	LIKELIHOOD	STATUS
RangoOftMiddleware.sol	Business Logic	High	High	Addressed

RangoOftMiddleware is a LayerZero compose receiver that processes incoming OFT messages. In `lzCompose()`, the function verifies that the composed message originates from a whitelisted OApp and that the caller is the OFT endpoint. However, there is **no validation** between the `_oApp` (OFT token contract) and the `bridgeToken` decoded from the interchain message (`m.bridgeRealOutput`). The whitelist only validates the OApp address, not the token it represents. In LayerZero's OFT model, each token has its own OApp contract. If the protocol holds high-value tokens in its contract and whitelists multiple OApps (e.g., USDT-OFT, USDC-OFT, WBTC-OFT), an attacker could craft a malicious cross-chain message where the `_oApp` is a whitelisted low-value token OApp (e.g., USDT-OFT) but `m.bridgeRealOutput` is set to a high-value token address (e.g., WBTC). Since the code passes `bridgeToken` (attacker-controlled) to `LibInterchainV2.handleDestinationMessage()`, the attacker can drain high-value tokens from the contract using a low-value OFT transfer.

RangoOftMiddleware::lzCompose()

```

67 function lzCompose(
68     address _oApp,
69     bytes32 /* _guid */,
70     bytes calldata _message,

```

```

71     address /* _executor */,
72     bytes calldata /* _extraData */
73 ) external payable override onlyWhenNotPaused nonReentrant {
74     RangoOftMiddlewareStorage storage s = getRangoOftMiddlewareStorage();
75     // Ensure the composed message comes from the correct OApp.
76     require(s.whitelistedOapps[_oApp], "ComposedReceiver: Invalid OApp");
77     require(msg.sender == s.oftEndpoint, "ComposedReceiver: Unauthorized sender");
78     ;
79     require(s.oftEndpoint != address(0), "OFT endpoint not initialized");
80     // Validate message length before slicing
81     require(_message.length >= COMPOSE_FROM_OFFSET, "Message too short");
82
83     bytes calldata rangoMessageBytes = _message[COMPOSE_FROM_OFFSET:];
84     Interchain.RangoInterChainMessage memory m = abi.decode((rangoMessageBytes),
85         (Interchain.RangoInterChainMessage));
86
87     // @dev using bridgeRealOutput to get the received token address is safe,
88     // because ETH (native) or WETH(weth) are not OFTs in any network.
89     address bridgeToken = m.bridgeRealOutput;
90     require(bridgeToken != address(0), "Invalid bridge token address");
91     uint256 amountLD = uint256(bytes32(_message[SRC_EID_OFFSET:AMOUNT_LD_OFFSET])
92         );
93
94     (address receivedToken, uint dstAmount, IRango2.CrossChainOperationStatus
95         status) = LibInterchainV2.handleDestinationMessage(bridgeToken, amountLD,
96         m);
97
98     ...
99 }

```

Remediation Maintain a mapping of `oApp => expectedToken` in `RangoOftMiddlewareStorage` and validate that the decoded `bridgeToken` matches the expected token for the given `_oApp`.

3.3.3 [M-1] Possible Revert in `LibSwapper::callSwapsAndFees()`

Target	Category	IMPACT	LIKELIHOOD	STATUS
LibSwapper.sol	Business Logic	Medium	Medium	Addressed

The `callSwapsAndFees()` function is designed to swap the user-specified `fromToken` into `toToken` on the source chain. It also handles charging a swap fee through the `collectFeesForSwap()` function (line 285). By design, this fee can be taken in either the `fromToken` or the `toToken`.

While examining its implementation, we observe that the fee is collected (line 285) before executing the token swap (line 296). If the fee is meant to be collected in the `toToken` (line 358), the transaction will be reverted. This is because the contract does not hold any `toToken` before the swap is performed.

LibSwapper::callSwapsAndFees()

```
281 function callSwapsAndFees(SwapRequest memory request, Call[] calldata calls)
    private returns (bytes[] memory) {
282     ...

284     // Get Fees
285     LibSwapper.collectFeesForSwap(request);

287     // Execute swap Calls
288     bytes[] memory returnData = new bytes[](calls.length);
289     address tmpSwapFromToken;
290     for (uint256 i = 0; i < calls.length; i++) {
291         tmpSwapFromToken = calls[i].swapFromToken;
292         bool isTokenNative = tmpSwapFromToken == ETH;
293         if (isTokenNative == false)
294             approveMax(tmpSwapFromToken, calls[i].spender, calls[i].amount);

296         (bool success, bytes memory ret) = isTokenNative
297         ? calls[i].target.call{value : calls[i].amount}(calls[i].callData)
298         : calls[i].target.call(calls[i].callData);

300         emit CallResult(calls[i].target, success, ret);
301         if (!success)
302             revert(_getRevertMsg(ret));
303         returnData[i] = ret;
304     }
305     ...
306 }
```

LibSwapper::collectFeesForSwap()

```
352 function collectFeesForSwap(SwapRequest memory request) internal {
353     BaseSwapperStorage storage baseSwapperStorage = getBaseSwapperStorage();
354     // Get Platform fee
355     bool hasPlatformFee = request.platformFee > 0;
356     bool hasDestExecutorFee = request.destinationExecutorFee > 0;
357     bool hasAffiliateFee = request.affiliateFee > 0;
358     address feeToken = request.feeFromInputToken ? request.fromToken : request.
        toToken;
359     if (hasPlatformFee hasDestExecutorFee) {
360         require(baseSwapperStorage.feeContractAddress != ETH, "Fee contract
            address not set");
361         _sendToken(feeToken, request.platformFee + request.
            destinationExecutorFee, baseSwapperStorage.feeContractAddress,
            feeToken == ETH, false);
362     }

364     // Get affiliate fee
365     if (hasAffiliateFee) {
```

```

366         require(request.affiliatorAddress != ETH, "Invalid affiliatorAddress");
367         _sendToken(feeToken, request.affiliateFee, request.affiliatorAddress,
368             feeToken == ETH, false);
369     }
370     ...
371 }

```

Remediation The `callSwapsAndFees()` function should be called after the swap operation has been completed.

3.3.4 [M-2] Improper Swap-out Amount in `onChainSwapsInternal()`

Target	Category	IMPACT	LIKELIHOOD	STATUS
LibSwapper.sol	Business Logic	Medium	Medium	Addressed

The `onChainSwapsInternal()` function is responsible for swapping the user-specified `fromToken` into `toToken` and returning the resulting amount of `toToken`. It performs the token swap and charges the swap fee by calling the `callSwapsAndFees()` function (line 247). Especially, the amount of `toToken` received (i.e., `secondaryBalance`) is calculated by checking the contract's `toToken` balance before and after the swap process (line 259).

While examining its implementation, it turns out that if the swap fee is taken in the `toToken`, it incorrectly subtracts the swap fee from the `secondaryBalance` (line 262). This is erroneous because `secondaryBalance` already accounts for the swap fee deduction, resulting in a double deduction.

```

LibSwapper::onChainSwapsInternal()

233 function onChainSwapsInternal(
234     SwapRequest memory request,
235     Call[] calldata calls,
236     uint256 extraNativeFee
237 ) internal returns (bytes[] memory, uint) {

239     uint toBalanceBefore = getBalanceOf(request.toToken);
240     uint fromBalanceBefore = getBalanceOf(request.fromToken);
241     uint256[] memory initialBalancesList = getInitialBalancesList(calls);

243     // transfer tokens from user for SwapRequest and Calls that require transfer
244     // from user.
245     transferTokensFromUserForSwapRequest(request);
246     transferTokensFromUserForCalls(calls);

247     bytes[] memory result = callSwapsAndFees(request, calls);

```

```

249     // check if any extra tokens were taken from contract and return excess
        tokens if any.
250     returnExcessAmounts(request, calls, initialBalancesList);

252     // get balance after returning excesses.
253     uint fromBalanceAfter = getBalanceOf(request.fromToken);

255     ...

257     uint toBalanceAfter = getBalanceOf(request.toToken);

259     uint secondaryBalance = toBalanceAfter - toBalanceBefore;
260     require(secondaryBalance >= request.minimumAmountExpected, "Output is less
        than minimum expected");

262     return (result, secondaryBalance - (request.feeFromInputToken ? 0 : sumFees(
        request)));
263 }

```

Remediation Adjust the calculation to prevent the double deduction and accurately reflect the correct amount of toToken.

3.3.5 [M-3] Revisited Logic of LibSwapper::onChainSwapsPreBridge()

Target	Category	IMPACT	LIKELIHOOD	STATUS
LibSwapper.sol	Business Logic	Medium	Medium	Addressed

The onChainSwapsPreBridge() function is designed to perform a token swap on the source chain before bridging. Within this function, the minimum amount of native token required is checked at lines 218-219. However, the calculation of minimumRequiredValue does not account for the scenario where toToken is used as the fee token. This oversight can lead to the minimum check triggering a revert in such cases.

LibSwapper::onChainSwapsPreBridge()

```

211 function onChainSwapsPreBridge(
212     SwapRequest memory request,
213     Call[] calldata calls,
214     uint extraFee
215 ) internal returns (uint out) {

217     bool isNative = request.fromToken == ETH;
218     uint minimumRequiredValue = (isNative ? request.platformFee + request.

```

```

        affiliateFee + request.amountIn + request.destinationExecutorFee : 0) +
        extraFee;
219     require(msg.value >= minimumRequiredValue, 'Send more ETH to cover input
        amount + fee');

221     (, out) = onChainSwapsInternal(request, calls, extraFee);
222     // when there is a bridge after swap, set the receiver in swap event to
        address(0)
223     emitSwapEvent(request, out, ETH);

225     return out;
226 }

```

Remediation When `request.feeFromInputToken==false`, the swap fee should not be added into `minimumRequiredValue`.

3.3.6 [M-4] Lack of Input Validation in `callRecieveMessage()`

Target	Category	IMPACT	LIKELIHOOD	STATUS
RangoCCTPV2Middleware.sol	Business Logic	High	Low	Addressed

The `callRecieveMessage()` function verifies the authenticity of cross-chain messages using `IMessageTransmitterV2(s.messageTransmitterV2).receiveMessage(message, signature)` (line 64). However, it fails to validate the user-provided `_mintToken` parameter against the expected token address (e.g., `burnToken`) specified in the decoded CCTP message body. This allows an attacker to initiate a valid USDC cross-chain transfer, obtain a legitimate message and signature, and call `callRecieveMessage(message, signature, WBTC_ADDRESS)` with an arbitrary token address. The function then uses `_mintToken` directly in `LibInterchain.handleDestinationMessage()` (lines 72 - 77) without further checks, whereby the attacker receives WBTC instead of the intended USDC, potentially causing significant financial loss or asset mismatch.

RangoCCTPV2Middleware::callRecieveMessage()

```

56 function callRecieveMessage(bytes calldata message, bytes calldata signature,
    address _mintToken)
57     external
58     nonReentrant
59     onlyWhenNotPaused
60 {
61     CCTPV2Storage storage s = getCCTPV2Storage();
62     // Call the receiveMessage function of the message transmitter
63     if (!IMessageTransmitterV2(s.messageTransmitterV2).receiveMessage(message,
        signature)) {

```

```

64         revert RangoCCTPV2Middleware__MessageTransmissionFailed();
65     }

67     CCTPV2Message memory decodedMessage = decodeMessage(message);
68     CCTPV2MessageBody memory decodedMessageBody = this.decodeMessageBody(
        decodedMessage.messageBody);

70     Interchain.RangoInterChainMessage memory m =
71         abi.decode((decodedMessageBody.hookData), (Interchain.
            RangoInterChainMessage));
72     (address receivedToken, uint256 dstAmount, IRango2.CrossChainOperationStatus
        status) = LibInterchain
73         .handleDestinationMessage(
74             _mintToken,
75             decodedMessageBody.amount - decodedMessageBody.feeExecuted,
76             m
77         );

79     emit RangoBridgeCompleted(
80         m.requestId, receivedToken, m.originalSender, m.recipient, dstAmount,
            status, m.dAppTag
81     );
82 }

```

Remediation Ensure that the user-provided `_mintToken` parameter matches the `burnToken` field in the decoded CCTP message body and validate that the contract receives the correct token amount.

3.3.7 [M-5] Reentrancy Guard Bypass Between Old and New Facets

Target	Category	IMPACT	LIKELIHOOD	STATUS
ReentrancyGuard.sol	Business Logic	Medium	Medium	Acknowledged

The `ReentrancyGuard` contract uses EIP-1153 transient storage (`tstore/tload`) for gas-efficient reentrancy protection. The `NAMESPACE` slot identifier is kept stable across upgrades, but transient storage and regular storage are two completely separate address spaces in the EVM. An older facet compiled with a previous version of `ReentrancyGuard` that used `SSTORE/SLOAD` will write to and read from regular storage, while newer facets using this version read from and write to transient storage. In a Diamond proxy architecture where both old and new facets coexist, a call sequence that enters through an old facet (setting the flag in regular storage) and then calls into a new facet will not see the reentrancy flag in transient storage. The new facet incorrectly believes it is not in a reentrant state and allows the call to proceed. If the new facet calls back into the old facet (or itself), reentrancy protection is bypassed. This creates a window where reentrancy attacks are possible during partial upgrades, which is particularly dangerous for facets handling token transfers, swaps, or

bridge operations.

ReentrancyGuard::nonReentrant()

```
10 abstract contract ReentrancyGuard {
11     bytes32 private constant NAMESPACE = keccak256("exchange.rango.
        reentrancyguard");

13     error ReentrancyError();
14     uint256 private constant _ENTERED = 1;

16     modifier nonReentrant() {
17         bytes32 slot = NAMESPACE;
18         uint256 status;
19         assembly {
20             status := tload(slot)
21         }
22         if (status == _ENTERED) revert ReentrancyError();
23         assembly {
24             tstore(slot, _ENTERED)
25         }
26         _;
27         assembly {
28             tstore(slot, 0)
29         }
30     }
31 }
```

Remediation Coordinate a full upgrade of all facets that use `nonReentrant` to the same implementation. Do not allow mixed `SSTORE`/`TSTORE` versions to coexist. Alternatively, abandon the transient storage optimization and revert to the `SSTORE`-based `ReentrancyGuard` to ensure backward compatibility across all facets.

3.3.8 [L-1] Potential Risks Associated with Centralization

Target	Category	IMPACT	LIKELIHOOD	STATUS
Multiple Contracts	Security	Low	Low	Acknowledged

In the `Rango V2.1` protocol, the existence of a privileged `owner` account introduces centralization risks, as it holds significant control and authority over critical operations governing the protocol. In the following, we show the representative function potentially affected by the privileges associated with the privileged account.

DiamondCutFacet::diamondCut()

```
7  contract DiamondCutFacet is IDiamondCut {
8      /// @notice Add/replace/remove any number of functions and optionally execute
9      ///         a function with delegatecall
10     /// @param _diamondCut Contains the facet addresses and function selectors
11     /// @param _init The address of the contract or facet to execute _calldata
12     /// @param _calldata A function call, including function selector and
13     ///         arguments
14     ///         _calldata is executed with delegatecall on _init
15     function diamondCut(
16         FacetCut[] calldata _diamondCut,
17         address _init,
18         bytes calldata _calldata
19     ) external override {
20         LibDiamond.enforceIsContractOwner();
21         LibDiamond.diamondCut(_diamondCut, _init, _calldata);
22     }
```

Remediation To mitigate the identified issue, it is recommended to introduce multi-sig mechanism to undertake the role of the privileged account. Moreover, it is advisable to implement timelocks to govern all modifications to the privileged operations.

Response By Team This issue has been confirmed by the team.

3.3.9 [L-2] Improper Event in _handleUniswapV2()/_handleCurve()

Target	Category	IMPACT	LIKELIHOOD	STATUS
LibInterchain.sol	Coding Practices	Low	Low	Addressed

Events are crucial for blockchain transparency, reliability, and interoperability. They play a vital role in updating user interfaces, confirming transactions, and facilitating cross-contract communication in decentralized applications (DApps). Incorrect event statuses can mislead users and systems that depend on these events.

While examining the implementation of `_handleUniswapV2()` (which is used to interact with `UniswapV2` for token exchange), we observe that it emits an `ActionDone` event with the `success` flag set to `true` regardless of whether the token exchange operation succeeds or fails. This behavior is misleading and incorrect, as it fails to accurately reflect the actual outcome of the exchange operation. Similarly, the `_handleCurve()` function exhibits the same issue.

```

LibInterchain::_handleUniswapV2()

144 try
145     IUniswapV2(action.dexAddress).swapExactTokensForTokens(
146         _amount,
147         action.amountOutMin,
148         action.path,
149         address(this),
150         action.deadline
151     )
152 returns (uint256[] memory) {
153     emit ActionDone(Interchain.ActionType.UNI_V2, action.dexAddress, true, "");
154     // Note: instead of using return amounts of swapExactTokensForTokens,
155     // we get the diff balance of before and after. This prevents errors
156     // for tokens with transfer fees
157     uint toBalanceAfter = LibSwapper.getBalanceOf(toToken);
158     SafeERC20.forceApprove(IERC20(action.path[0]), action.dexAddress, 0);
159     return (true, toBalanceAfter - toBalanceBefore, toToken);
160 } catch {
161     emit ActionDone(Interchain.ActionType.UNI_V2, action.dexAddress, true, "
162         Uniswap-V2 call failed");
163     SafeERC20.forceApprove(IERC20(action.path[0]), action.dexAddress, 0);
164     return (false, _amount, shouldDeposit ? weth : _token);
165 }

```

Remediation Improve the ActionDone event in the _handleUniswapV2()/_handleCurve() functions.

3.3.10 [L-3] Revisited Logic of LibSwapper2::returnExcessAmounts()

Target	Category	IMPACT	LIKELIHOOD	STATUS
LibSwapper2.sol	Business Logic	Low	Medium	Addressed

The returnExcessAmounts() function calculates excess amounts by comparing current balances with initial balances for each swapToToken, but it fails to account for the extraNativeFee parameter that is specifically reserved for cross-chain bridge operations. When ETH is an intermediate token in the swap process, the function incorrectly returns all excess ETH (including the bridge fee portion) to users, leaving insufficient ETH for bridge transaction costs and causing cross-chain operations to fail.

```

LibSwapper2::onChainSwapsInternal()

223 function onChainSwapsInternal(
224     SwapRequest memory request,
225     Call[] calldata calls,
226     uint256 extraNativeFee

```

```

227 ) internal returns (bytes[] memory, uint) {
228     uint toBalanceBefore = getBalanceOf(request.toToken);
229     uint fromBalanceBefore = getBalanceOf(request.fromToken);
230     uint256[] memory initialBalancesList = getInitialBalancesList(calls);

232     // transfer tokens from user for SwapRequest and Calls that require transfer
        from user.
233     transferTokensFromUserForSwapRequest(request);
234     transferTokensFromUserForCalls(calls);

236     bytes[] memory result = callSwapsAndFees(request, calls);

238     returnExcessAmounts(request, calls, initialBalancesList);
239     ...
240 }

```

LibSwapper2::returnExcessAmounts()

```

559 function returnExcessAmounts(
560     SwapRequest memory request,
561     Call[] calldata calls,
562     uint256[] memory initialBalancesList) internal {
563     uint excessAmountToToken;
564     address tmpSwapToToken;
565     uint currentBalanceTo;
566     for (uint256 i = 0; i < calls.length; i++) {
567         tmpSwapToToken = calls[i].swapToToken;
568         currentBalanceTo = getBalanceOf(tmpSwapToToken);
569         excessAmountToToken = currentBalanceTo - initialBalancesList[i];
570         if (excessAmountToToken > 0 && tmpSwapToToken != request.toToken) {
571             _sendToken(tmpSwapToToken, excessAmountToToken, msg.sender);
572         }
573     }
574 }

```

Remediation Add the `extraNativeFee` parameter to the `returnExcessAmounts()` function and improve the implementation to reserve the specified amount of ETH for bridge transaction fees when ETH is an intermediate token.

3.3.11 [L-4] Incompatible msg.sender Fallbacks with Permit2Proxy Call Path

Target	Category	IMPACT	LIKELIHOOD	STATUS
Multiple Contracts	Business Logic	Low	Medium	Acknowledged

In the Diamond, `msg.sender` is used as the default user address in numerous places (e.g., `excess`

token refunds, bridge depositor/refund fallback, EOA validation). When the call path goes through Permit2Proxy, `msg.sender` becomes the Proxy contract address instead of the real user. This causes excess swap tokens and bridge refunds to be sent to Permit2Proxy rather than the user, and makes the Thorchain EOA check unconditionally revert since Permit2Proxy is a contract.

LibSwapper2::onChainSwapsInternal()

```
223 function onChainSwapsInternal(  
224     SwapRequest memory request,  
225     Call[] calldata calls,  
226     uint256 extraNativeFee  
227 ) internal returns (bytes[] memory, uint) {  
228     ...  
  
230     // check over-expense of fromToken and return excess if any.  
231     if (request.fromToken != ETH) {  
232         require(fromBalanceAfter >= fromBalanceBefore, "Source token balance on  
233             contract must not decrease after swap");  
234         if (fromBalanceAfter > fromBalanceBefore)  
235             _sendToken(request.fromToken, fromBalanceAfter - fromBalanceBefore,  
236                 msg.sender);  
237     }  
238     else {  
239         require(fromBalanceAfter >= fromBalanceBefore - msg.value +  
240             extraNativeFee, "Source token balance on contract must not decrease  
241             after swap");  
242         // When we are keeping extraNativeFee for bridgingFee, we should consider  
243             it in calculations.  
244         if (fromBalanceAfter > fromBalanceBefore - msg.value + extraNativeFee)  
245             _sendToken(request.fromToken, fromBalanceAfter + msg.value -  
246                 fromBalanceBefore - extraNativeFee, msg.sender);  
247     }  
  
248     uint toBalanceAfter = getBalanceOf(request.toToken);  
249     uint secondaryBalance = toBalanceAfter - toBalanceBefore;  
250     require(secondaryBalance >= request.minimumAmountExpected, "Output is less  
251         than minimum expected");  
  
252     return (result, secondaryBalance);  
253 }
```

LibSwapper2::returnExcessAmounts()

```
557 function returnExcessAmounts(  
558     SwapRequest memory request,  
559     Call[] calldata calls,  
560     uint256[] memory initialBalancesList) internal {
```

```
561     uint excessAmountToToken;
562     address tmpSwapToToken;
563     uint currentBalanceTo;
564     for (uint256 i = 0; i < calls.length; i++) {
565         tmpSwapToToken = calls[i].swapToToken;
566         currentBalanceTo = getBalanceOf(tmpSwapToToken);
567         excessAmountToToken = currentBalanceTo - initialBalancesList[i];
568         if (excessAmountToToken > 0 && tmpSwapToToken != request.toToken) {
569             _sendToken(tmpSwapToToken, excessAmountToToken, msg.sender);
570         }
571     }
572 }
```

Remediation If Permit2Proxy is retained as a supported entrypoint, do not use `msg.sender` as a default user address anywhere in the Diamond. All functions that currently fall back to `msg.sender` for refunds, depositors, delegates, or refund recipients must require the user to explicitly provide these addresses in the request struct, and the transaction should revert if they are not supplied. Note that RangoThorchainFacet is inherently incompatible with Permit2Proxy when `preventNonEoaSender` is enabled, as Permit2Proxy is a contract and will always fail the EOA check.

4 | Appendix

4.1 About AstraSec

AstraSec is a blockchain security company that serves to provide high-quality auditing services for blockchain-based protocols. With a team of blockchain specialists, AstraSec maintains a strong commitment to excellence and client satisfaction. The audit team members have extensive audit experience for various famous DeFi projects. AstraSec's comprehensive approach and deep blockchain understanding make it a trusted partner for the clients.

4.2 Disclaimer

The information provided in this audit report is for reference only and does not constitute any legal, financial, or investment advice. Any views, suggestions, or conclusions in the audit report are based on the limited information and conditions obtained during the audit process and may be subject to unknown risks and uncertainties. While we make every effort to ensure the accuracy and completeness of the audit report, we are not responsible for any errors or omissions in the report.

We recommend users to carefully consider the information in the audit report based on their own independent judgment and professional advice before making any decisions. We are not responsible for the consequences of the use of the audit report, including but not limited to any losses or damages resulting from reliance on the audit report.

This audit report is for reference only and should not be considered a substitute for legal documents or contracts.

4.3 Contact

Phone	+86 156 0639 2692
Email	contact@astrasec.ai
Twitter	https://twitter.com/AstraSecAI